Subject: Full-Stack Development

Subject Code: 24CSP-210

Project Report: Job Portal

Submitted by:
Student Name: Ranvir Singh
UID: 24BCS10619
Branch: BE:CSE
Section/Group: 24BCS_KRG_616-B

Submitted to:
Mr. Deep Prakash Gupta

Job Portal Web Application

1. Overview

The Job Portal Web Application is a full-stack web-based system developed using React.js for the frontend and Spring Boot (Java) for the backend, with MySQL as the database.

The main objective of this project is to create a platform where users can:

- Register and login securely
- View available job opportunities
- Add job listings

This project demonstrates the implementation of a client-server architecture, where the frontend communicates with the backend through REST APIs.

2. Objectives

- To build a full-stack web application
- To understand frontend-backend integration
- To implement CRUD operations using REST APIs
- To design a responsive user interface
- To use database connectivity with JPA

3. Technologies Used

Frontend:

- React.js
- HTML, CSS
- Axios (API communication)

Backend:

- Java
- Spring Boot
- Spring Data JPA

Database:

- MySQL

Tools:

- VS Code
- Postman / Thunder Client
- Maven

4. Features of the Project

- User Registration
- User Login Authentication
- Add Job Functionality
- View Job Listings
- Dashboard with job statistics
- Responsive UI design
- Navigation between pages

5. Functionality

1. Registration

- User enters email and password
- Data is sent to backend via API
- Stored in MySQL database

2. Login

- User credentials are verified
- If valid → redirected to dashboard
- If invalid → error message shown

3. Add Job

- User enters job title and description
- Data stored in database

4. View Jobs

- Jobs fetched from backend
- Displayed dynamically in frontend

5. Dashboard

- Shows total jobs
- Displays recent job listings

6. Project Architecture

This project follows a three-tier architecture:

1. Presentation Layer (Frontend)

- Built using React
- Handles UI and user interaction

2. Application Layer (Backend)

- Spring Boot handles business logic
- REST APIs process requests

3. Data Layer (Database)

- MySQL stores user and job data

7. Flow of the Project

Step-by-step flow:

- User opens the web application
- User registers or logs in
- React frontend sends request using Axios
- Spring Boot backend receives request
- Controller processes request
- Repository interacts with database
- Data is stored/fetched from MySQL
- Response sent back to frontend
- UI updates dynamically

8. API Endpoints

Method	Endpoint	Description
POST	/register	Register user
POST	/login	Login user
POST	/add-job	Add new job
GET	/jobs	Get all jobs

9. Database Design

User Table:

- id (Primary Key)
- email
- password

Job Table:

- id (Primary Key)
- title
- description

10. User Interface Design

- Modern login & register page
- Dashboard with statistics
- Job cards layout
- Responsive design using CSS

11. Challenges Faced

- API connection errors
- CORS issues
- Database configuration problems
- UI responsiveness issues

12. Future Enhancements

- Apply to job feature
- User profile system
- Admin panel
- Job search and filter
- Email notifications

13. Conclusion

The project successfully demonstrates a full-stack web application integrating frontend, backend, and database. It provides a basic but functional job portal system and helps in understanding real-world web development concepts.